

CSC-501

TCS NOTES

Module 1:
Basic Concepts and Finite
Automata

Prof. Kranti M. Gajmal
Computer Engineering Dept.
Gharda Institute of Technology

Automata – What is it?

In theoretical computer science, the theory of computation is the branch that deals with whether and how efficiently problems can be solved on a model of computation, using an algorithm. The field is divided into three major branches: automata theory, computability theory and computational complexity theory.

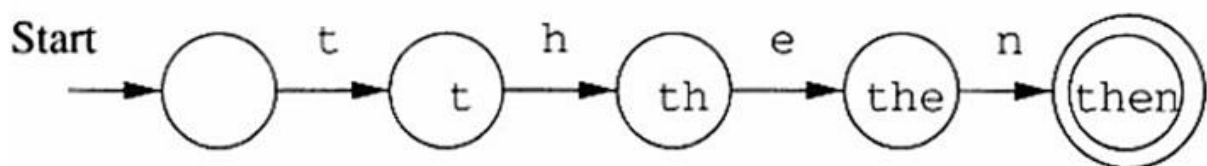
The term "Automata" is derived from the Greek word "αὐτόματα" which means "self-acting". An automaton (Automata in plural) is an abstract self-propelled computing device which follows a predetermined sequence of operations automatically.

An automaton with a finite number of states is called a **Finite Automaton** (FA) or **Finite State Machine** (FSM).

This automaton consists of

- states (represented in the figure by circles),
- and transitions (represented by arrows).

As the automaton sees a symbol of input, it makes a transition (or jump) to another state, according to its transition function (which takes the current state and the recent symbol as its inputs).



Uses of Automata: compiler design and parsing.

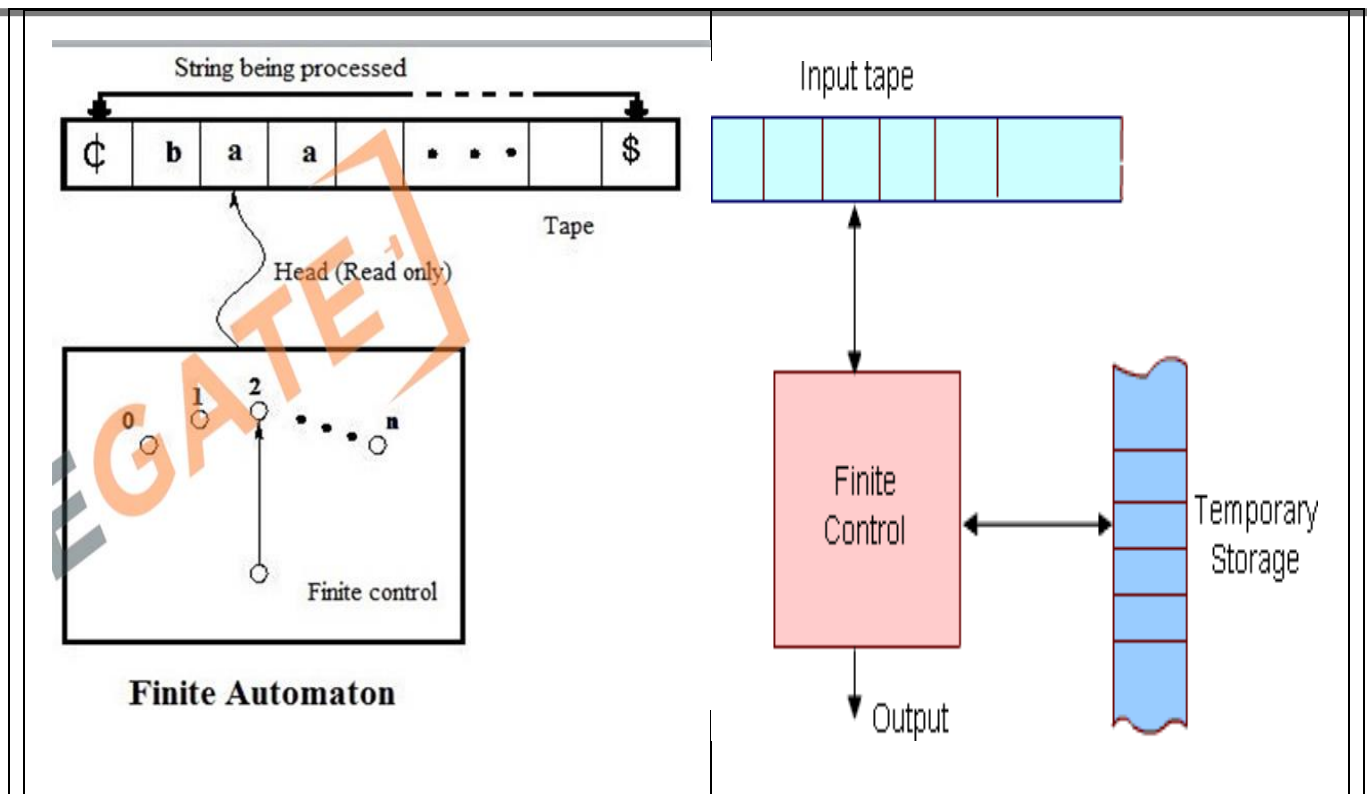
Components/Block Diagram/Model of Finite Automata/ Finite State Machine

The various components of the block diagram are explained as follows:

●**Input tape:** The input tape is divided into squares, each square containing a single symbol from the input alphabet Σ . The end squares of the tape contain the end marker ϕ at the left end and the end marker $\$$ at the right end. The absence of end markers indicates that the tape is of infinite length. The left-to-right sequence of symbols between the two end markers is the input string to be processed.

●**Reading head (R-head):** The head examines only one square at a time and will move one square to the right.

●**Finite control:** is the inference engine take care of transition



The figure above shows a diagrammatic representation of generic finite automata.

Operation of the automation is defined as follows.

- At any point of time the automaton is in some integral state and is reading a particular symbol from the input tape by using the mechanism for reading input.
- In the next time step the automaton then moves to some other integral (or remain in the same state) as defined by the transition function. The transition function is based on the current state, input symbol read, and the content of the temporary storage.
- At the same time the content of the storage may be changed and the input read may be modified. The automation may also produce some output during this transition. The internal state, input and the content of storage at any point defines the configuration of the automaton at that point.
- The transition from one configuration to the next (as defined by the transition function) is called a move.
- Finite state machine or Finite Automation is the simplest type of abstract machine we consider. Any system that is at any point of time in one of a finite number of interval state and moves among these states in a defined manner in response to some input, can be modeled by a finite automaton.
- It does not have any temporary storage and hence a restricted model of computation.

Formal definition of a Finite Automaton

An automaton can be represented by a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where:

- **Q** is a finite set of states.
- **Σ** is a finite set of symbols, called the **alphabet** of the automaton.
- **δ** is the transition function.
- **q_0** is the initial state from where any input is processed ($q_0 \in Q$).
- **F** is a set of final state/states of Q ($F \subseteq Q$).

Related Terminologies

SYMBOL- Symbols are the basic building blocks, which can be any character/token. (cow, sheep, white flag, etc.) (in English we called them as letters).

Alphabet

- **Definition:** An **alphabet** is any finite set of symbols.
- **Example:** $\Sigma = \{a, b, c, d\}$ is an **alphabet set** where 'a', 'b', 'c', and 'd' are **symbols**.

String

- **Definition:** A **string** is a finite sequence of symbols taken from Σ .
- **Example:** 'cabcad' is a valid string on the alphabet set $\Sigma = \{a, b, c, d\}$

Length of a String

- **Definition :** It is the number of symbols present in a string. (Denoted by $|S|$).

Examples:

If $S = \text{'cabcad'}$, $|S| = 6$

If $|S| = 0$, it is called an empty string (Denoted by λ or ϵ)

Kleene Star

- **Definition:** The Kleene star, Σ^* , is a unary operator on a set of symbols or strings, Σ , that gives the infinite set of all possible strings of all possible lengths over Σ including ϵ .
- **Representation:** $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots$ where Σ^p is the set of all possible strings of length p.

- Example: If $\Sigma = \{a, b\}$, $\Sigma^* = \{\lambda, a, b, aa, ab, ba, bb, \dots\}$

Kleene Closure / Plus

- Definition: The set Σ^+ is the infinite set of all possible strings of all possible lengths over Σ excluding ϵ .

- Representation: $\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$
 $\Sigma^+ = \Sigma^* - \{\epsilon\}$

- Example: If $\Sigma = \{a, b\}$, $\Sigma^+ = \{a, b, aa, ab, ba, bb, \dots\}$

Language

- Definition: A language is a subset of Σ^* for some alphabet Σ . It can be finite or infinite.
- Example: If the language takes all possible strings of length 2 over $\Sigma = \{a, b\}$, then $L = \{ab, bb, ba, aa\}$
- A language is defined as a set of strings. (in natural language (set of words(predefined) and grammar) we apply this model from words to sentence).
- So, we may have languages that have infinite number of words, so it is not possible for us to list them, we have to use some framework, which can somehow represent the same language.

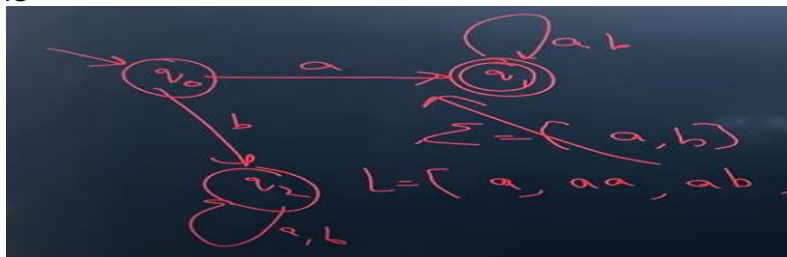
There are mainly two methods to represent a language

- By a grammar that generates a language [Regular Grammar generate Regular Language]
- By a machine that accepts a language [FA accept Regular language]

Language contains the string having first symbol a

$\Sigma = \{a, b\}$
 $L = \{a, aa, ab, aaa, aaaa, \dots\}$

Machine



Grammar

$S \rightarrow aA$
 $A \rightarrow aA / bA / \epsilon$

Finite automata can be broadly classified into two types-

1) Finite automata without output

1. Deterministic finite automata.
2. Non deterministic finite automata.
3. Non deterministic finite automata with ϵ

2) Finite automata with output

1. Moore machine
2. Mealy machine Finite Automaton can be classified into two types:

Deterministic Finite Automaton (DFA)

In DFA, for each input symbol, one can determine the state to which the machine will move. Hence, it is called **Deterministic Automaton**. As it has a finite number of states,

the machine is called **Deterministic Finite Machine** or **Deterministic Finite Automaton**.

Formal Definition of a DFA

A DFA can be represented by a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where:

- Q is a finite set of states.
- Σ is a finite set of symbols called the alphabet.
- δ is the transition function where $\delta: Q \times \Sigma \rightarrow Q$
- q_0 is the initial state from where any input is processed ($q_0 \in Q$).
- F is a set of final state/states of Q ($F \subseteq Q$).

Graphical Representation of a DFA

A DFA is represented by digraphs called **state diagram**.

- The vertices represent the states.
- The arcs labeled with an input alphabet show the transitions.
- The initial state is denoted by an empty single incoming arc.
- The final state is indicated by double circles.

Transition table representation of DFA

TRANSITION TABLE- it is a two-dimensional table where number of columns is equal to number of input alphabets and number of rows is equal to number of states.

Example

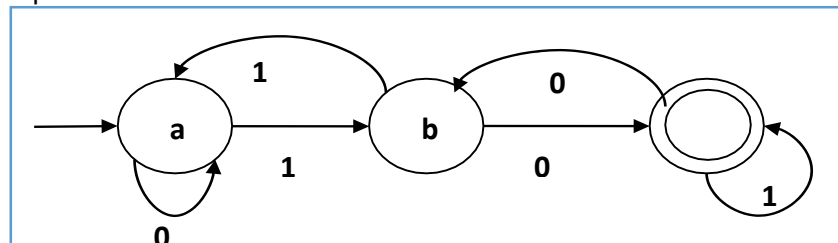
Let a deterministic finite automaton be $\square$

- $Q = \{a, b, c\}$,
- $\Sigma = \{0, 1\}$,
- $q_0 = \{a\}$,
- $F = \{c\}$, and

Transition function δ as shown by the following table:

Present State	Next State for Input 0	Next State for Input 1
a	a	b
b	c	a
c	b	c

Its graphical representation would be as follows:



DFA – Graphical Representation

ACCEPTANCE BY DFA

• Let 'w' be any string designed from the alphabet, corresponding to w, if there exist a transition for which it starts at the initial state and ends in any One of the final states, then the string 'w' is said to be accepted by the finite automata. $\delta^*(q_0, w) = q_f$ for some $q_f \in F$.

• Mathematically, it can be represented as: - $L(M) = \{w \in \Sigma^* \mid \delta^*(S, w) \in F\}$

Note

- DFA Produces a unique computation (or run) of the automaton for each input string.
- Deterministic refers to the uniqueness of the computation.
- DFAs are useful for doing lexical analysis (Spell check) in compiler design.

Non Deterministic Finite Automaton (NFA)

On the input symbol, the machine can move to any combination of the states in the machine. In other words, the exact state to which the machine moves cannot be determined. Hence, it is called **Non-deterministic Automaton**. As it has finite number of states, the machine is called **Non-deterministic Finite Machine** or **Nondeterministic Finite Automaton**.

Formal Definition of an NDFA

An NDFA can be represented by a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where:

- **Q** is a finite set of states.
- **Σ** is a finite set of symbols called the alphabets.
- **δ** is the transition function where $\delta: Q \times \Sigma \rightarrow 2^Q$
(Here the power set of Q (2^Q) has been taken because in case of NDFA, from a state, transition can occur to any combination of Q states)
- **q_0** is the initial state from where any input is processed ($q_0 \in Q$).
- **F** is a set of final state/states of Q ($F \subseteq Q$).

Graphical Representation of an NDFA: (same as DFA)

An NDFA is represented by digraphs called state diagram.

- The vertices represent the states.
- The arcs labeled with an input alphabet show the transitions.
- The initial state is denoted by an empty single incoming arc.
- The final state is indicated by double circles.

Example

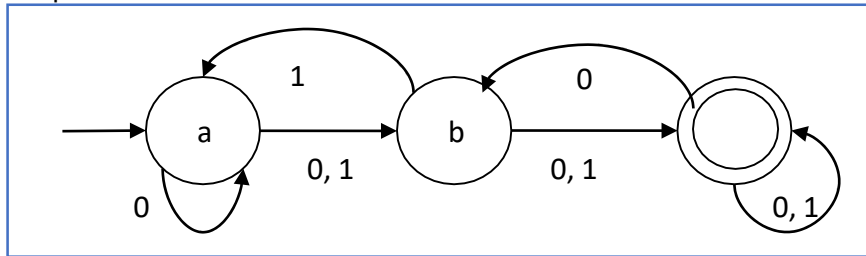
Let a non-deterministic finite automaton be

- $Q = \{a, b, c\}$
- $\Sigma = \{0, 1\}$
- $q_0 = \{a\}$
- $F = \{c\}$

The transitions function is shown below:

Present State	Next State for Input 0	Next State for Input 1
a	a, b	b
b	c	a, c
c	b, c	c

Its graphical representation would be as follows:



NDFA – Graphical Representation

DFA vs NDFA

The following table lists the differences between DFA and NDFA.

DFA	NDFA
The transition from a state is to a single particular next state for each input symbol. Hence it is called <i>deterministic</i> .	The transition from a state can be to multiple next states for each input symbol. Hence it is called <i>non-deterministic</i> .
Empty string transitions are not seen in DFA.	NDFA permits empty string transitions.
Backtracking is allowed in DFA	In NDFA, backtracking is not always possible.
Requires more space.	Requires less space.
A string is accepted by a DFA, if it transits to a final state.	A string is accepted by a NDFA, if at least one of all possible transitions ends in a final state.

Some points to remember

- Every NFA can be translated to an equivalent DFA, so their language accepting capability is same.

- NFAs like DFA's only recognize regular languages.
- It needs not to be a complete system. There can be a state that doesn't have any transition on some input symbol.
- It is possible that a single state led to multiple transitions on same input to different states.
- NOTE- A null transition is also possible for NFA, such special NFA are called Null-NFA. We will discuss it later.
- Accepting power of NDFA= Accepting power of DFA.
- NDFA is a theoretical engine and is not implementable, but it is very easy to design compare to DFA.
- No concept of dead state, therefore complementation of DFA is also not possible.
- NDFA will respond for only valid strings and no need to respond for invalid strings.
(it is a Incomplete system)

ACCEPTANCE BY NDFA

- Let 'w' be any string defined over the alphabet Σ , corresponding to w, there can be multiple transitions for NFA starting from initial state, if there exist at least one transition for which we start at the initial state and ends in any One of the final state, then the string 'w' is said to be accepted by the non-deterministic finite automata, otherwise not.

- Mathematically, it can be represented as, $L(M) = \{w \in \Sigma^* \mid \delta^*(q_0, w) \in F\}$

Acceptors, Classifiers, and Transducers

Acceptor (Recognizer)

An automaton that computes a Boolean function is called an **acceptor**. All the states of an acceptor is either accepting or rejecting the inputs given to it.

Classifier

A **classifier** has more than two final states and it gives a single output when it terminates.

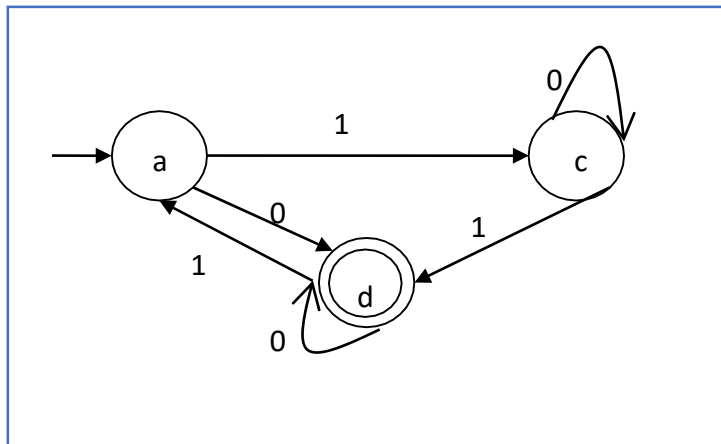
Transducer

An automaton that produces outputs based on current input and/or previous state is called a **transducer**. Transducers can be of two types:

- **Mealy Machine** The output depends both on the current state and the current input.
- **Moore Machine** The output depends only on the current state.

Example

Let us consider the DFA shown in Figure 1.3. From the DFA, the acceptable strings can be derived.



Acceptability of strings by DFA

Strings accepted by the above DFA: {0, 00, 11, 010, 101,}

Strings not accepted by the above DFA: {1, 011, 111, }

NDFA to DFA Conversion

Problem Statement

Let $X = (Q_x, \Sigma, \delta_x, q_0, F_x)$ be an NDFA which accepts the language $L(X)$. We have to design an equivalent DFA $Y = (Q_y, \Sigma, \delta_y, q_0, F_y)$ such that $L(Y) = L(X)$. The following procedure converts the NDFA to its equivalent DFA:

Algorithm 1

Input: An NDFA

Output: An equivalent DFA

Step 1 Create state table from the given NDFA.

Step 2 Create a blank state table under possible input alphabets for the equivalent DFA.

Step 3 Mark the start state of the DFA by q_0 (Same as the NDFA).

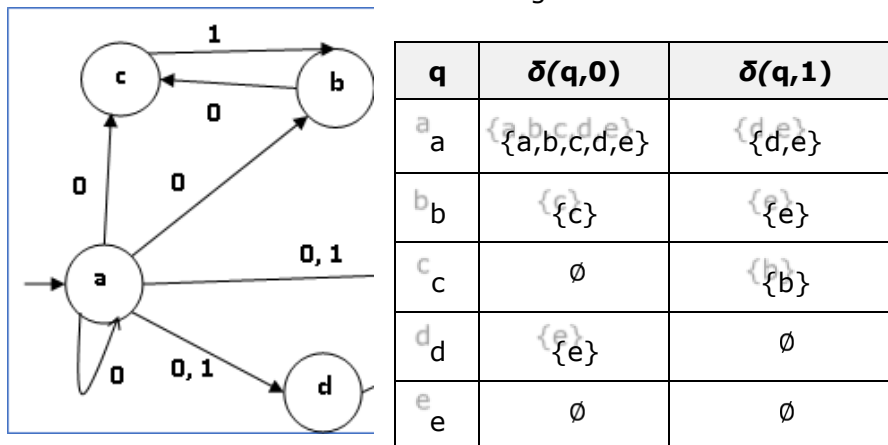
Step 4 Find out the combination of States $\{Q_0, Q_1, \dots, Q_n\}$ for each possible input alphabet.

Step 5 Each time we generate a new DFA state under the input alphabet columns, we have to apply step 4 again, otherwise go to step 6.

Step 6 The states which contain any of the final states of the NDFA are the final states of the equivalent DFA.

Example

Let us consider the NDFA shown in the figure below.



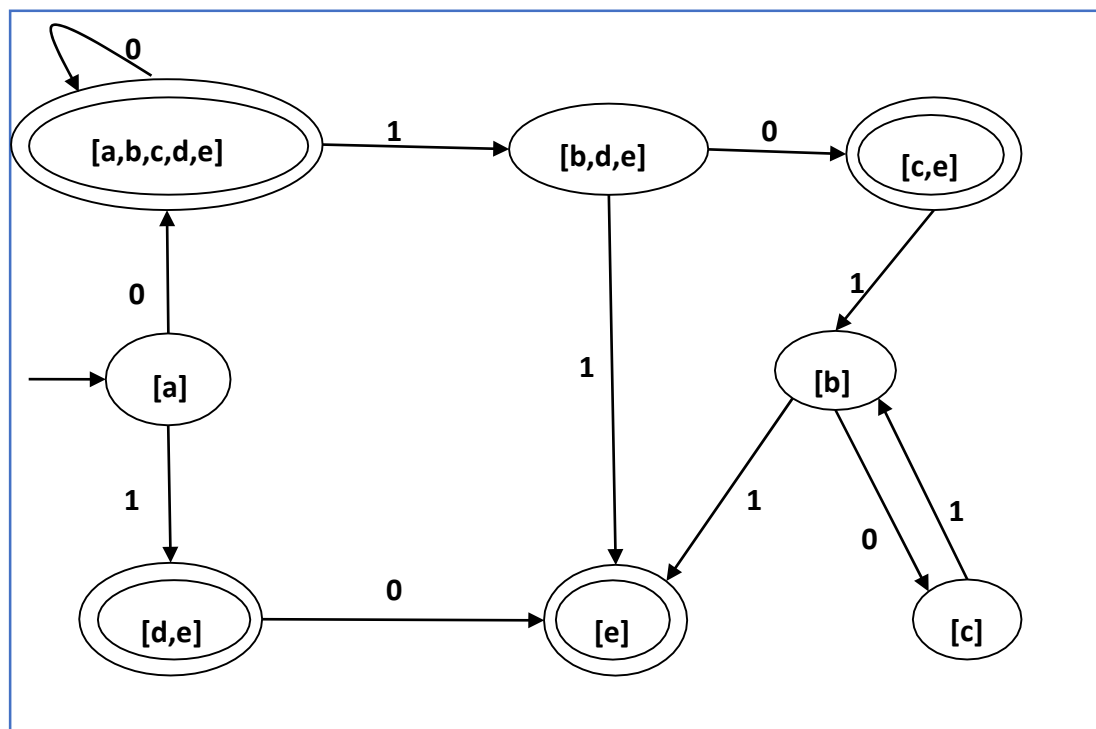
Using the above algorithm, we find its equivalent DFA. The state table of the DFA is shown in below.

q	$\delta(q,0)$	$\delta(q,1)$
[a]	[a,b,c,d,e]	[d,e]

[a,b,c,d,e]	[a,b,c,d,e]	[b,d,e]
[d,e]	[e]	$\emptyset$
[b,d,e]	[c,e]	[e]
[e]	$\emptyset$	$\emptyset$
[c,e]	$\emptyset$	[b]
[b]	[c]	[e]
[c]	$\emptyset$	[b]

State table of DFA equivalent to NFA

The state diagram of the DFA is as follows:

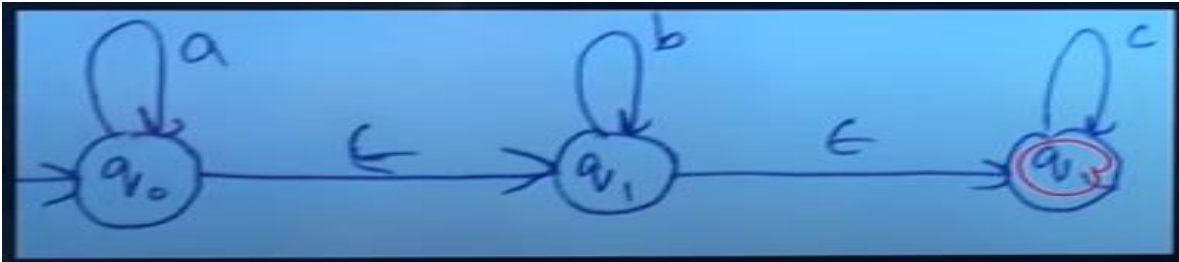


State diagram of DFA

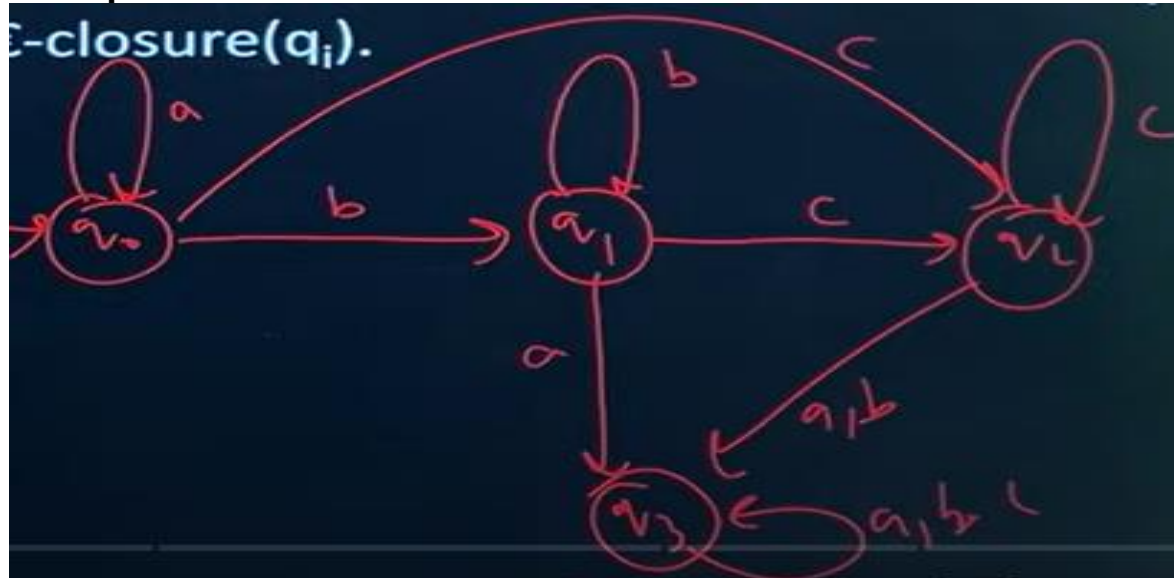
NFA WITH EPSILON MOVES (ϵ -NFA)

- An automaton that consists of null transitions is called a Null- NFA i.e. we allow a transition on null means empty string.
- ϵ -NFA is a 5-tuple $(Q, \Sigma, \delta, S, F)$ where:
- Q is a finite and non-empty set of states
- Σ is a finite non-empty set of finite input alphabet
- δ is a transition function $\delta: (Q \times \{\Sigma \cup \epsilon\}) \rightarrow 2^Q$

- S is initial state (always one) ($S \in Q$)
- F is a set of final states ($F \subseteq Q$) ($0 < |F| \leq N$, where n is the number of states)



It's Equivalent DFA



NULL-CLOSURE

- Null closure of a set Q is defined as a set of all the states, which are at zero distance from the state Q. A set of all the states that can be reached from the state and along a null- transition.
- ϵ -Closure(q_i)- The set of all the states which are at zero distance from the state q_i is called ϵ -closure(q_i). Or the set of all the states that can be reached from the state q_i along ϵ labeled transition path, is known as ϵ -closure(q_i).
- Every state is at zero distance to itself.
- In NFA or DFA, distance between two states is always 1, because there could be no null transitions.
- The Null closure Q is always a non-empty and finite state, because every state's null closure is that state only.
- ϵ -closure(Φ) = Φ
- ϵ -closure ($q_0, q_1, q_2, q_3, \dots, q_n$) = $\bigcup_{i=1}^n \epsilon$ -closure(q_i)

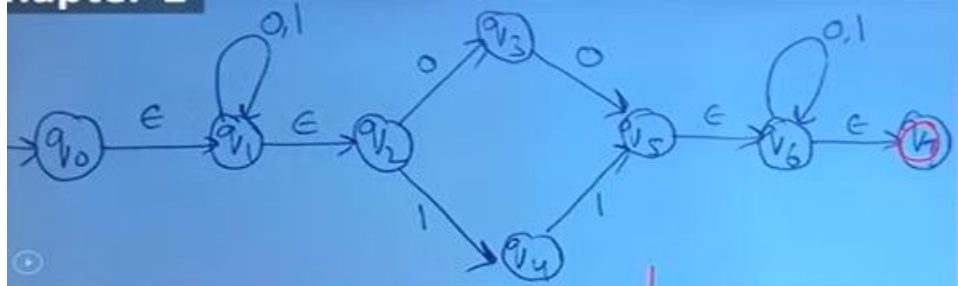
EQUIVALENCE BETWEEN NULL NFA TO NFA

- There will be no change in the initial state.
- No change in the total no. of states
- May be change in the number of final states.
- All the states will get the status of the final state in the resulting NFA, whose ϵ -closure contains at least one final state in the initial ϵ -NFA.

Steps to calculate NFA from ϵ - NFA

- Let us check for $\delta(q_0, a) = \epsilon\text{-Closure}[\delta[\epsilon\text{-Closure}(q_0), a]]$
- first, we find $\epsilon\text{-Closure}$ of q_0 : $\{q_0, q_1, q_2\}$
- Now check transition of all three states on input symbol a we get: q_0
- After that we again calculate the $\epsilon\text{-Closure}$ of the above result q_0 , we get the result as: $\{q_0, q_1, q_2\}$
- Similarly, we will check it for every state.
- All those states will be considered as a final state in which we have q_2 (final state in initial ϵ - NFA) in there $\epsilon\text{-Closure}$, i.e. all states will be final in this case.

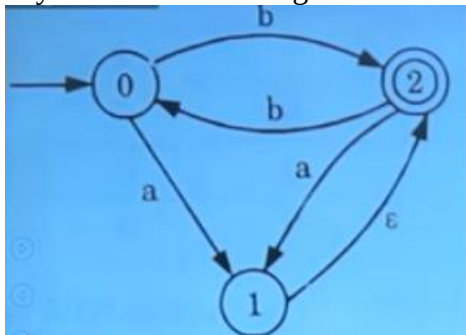
Example ϵ - NFA



Resultant NFA

	0	1
$\rightarrow q_0$	q_1, q_2, q_5	q_1, q_2, q_4
q_1	q_1, q_2, q_3	q_1, q_2, q_4
q_2	q_3	q_4
q_3	q_5, q_6, q_7	ϕ
q_4	ϕ	q_5, q_6, q_7
q_5	q_6, q_7	q_6, q_7
q_6	q_6, q_7	q_6, q_7
q_7	ϕ	ϕ

Try NFA for Following ϵ - NFA



DFA Minimization

DFA Minimization using Myhill-Nerode Theorem

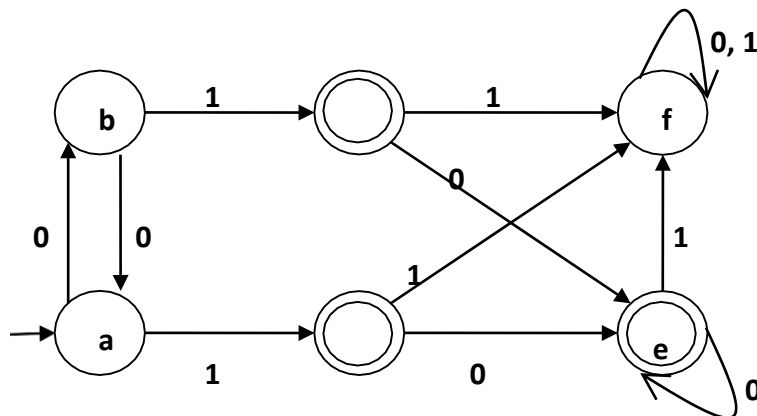
Algorithm 2

- Input** DFA Given
Output Minimized DFA
- Step 1** Draw a table for all pairs of states (Q_i, Q_j) not necessarily connected directly [All are unmarked initially]
- Step 2** Consider every state pair (Q_i, Q_j) in the DFA where $Q_i \in F$ and $Q_j \notin F$ or vice versa and mark them. [Here F is the set of final states]
- Step 3** Repeat this step until we cannot mark anymore states:

If there is an unmarked pair (Q_i, Q_j), mark it if the pair $\{\delta(Q_i, A), \delta(Q_j, A)\}$ is marked for some input alphabet.
- Step 4** Combine all the unmarked pair (Q_i, Q_j) and make them a single state in the reduced DFA.

Example

Let us use Algorithm 2 to minimize the DFA shown below.



State Diagram of DFA

Step 1 : We draw a table for all pair of states.

	a	b	c	d	e	f
a						
b						
c						
d						
e						
f						

Step 2 : We mark the state pairs:

	a	b	c	d	e	f
a						
b						
c	✓	✓				
d	✓	✓				
e	✓	✓				
f			✓	✓	✓	

Step 3 : We will try to mark the state pairs, with green colored check mark, transitively. If we input 1 to state 'a' and 'f', it will go to state 'c' and 'f' respectively. (c, f) is already marked, hence we will mark pair (a, f). Now, we input 1 to state 'b' and 'f'; it will go to state 'd' and 'f' respectively. (d, f) is already marked, hence we will mark pair (b, f).

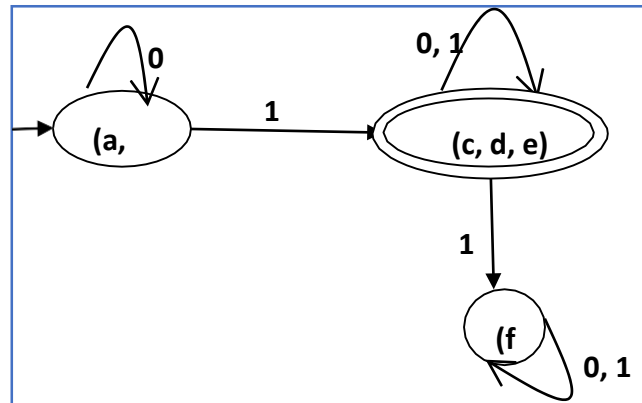
	a	b	c	d	e	f
a						
b						
c	✓	✓				
d	✓	✓				
e	✓	✓				
f	✓	✓	✓	✓	✓	

After step 3, we have got state combinations {a, b} {c, d} {c, e} {d, e} that are unmarked.

We can recombine {c, d} {c, e} {d, e} into {c, d, e}

Hence we got two combined states as: {a, b} and {c, d, e}

So the final minimized DFA will contain three states $\{f\}$, $\{a, b\}$ and $\{c, d, e\}$



State diagram of reduced DFA

DFA Minimization using Equivalence Theorem

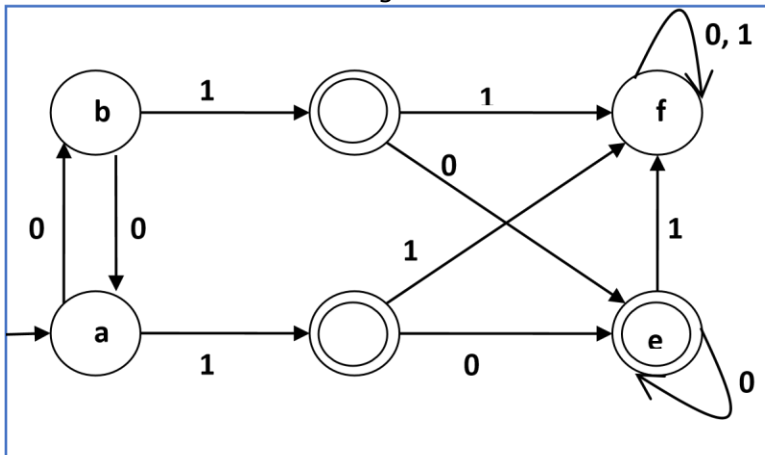
If X and Y are two states in a DFA, we can combine these two states into $\{X, Y\}$ if they are not distinguishable. Two states are distinguishable, if there is at least one string S , such that one of $\delta(X, S)$ and $\delta(Y, S)$ is accepting and another is not accepting. Hence, a DFA is minimal if and only if all the states are distinguishable.

Algorithm 3

- Step 1:** All the states Q are divided in two partitions: **final states** and **non-final states** and are denoted by P_0 . All the states in a partition are 0^{th} equivalent. Take a counter k and initialize it with 0.
- Step 2:** Increment k by 1. For each partition in P_k , divide the states in P_k into two partitions if they are k -distinguishable. Two states within this partition X and Y are k -distinguishable if there is an input S such that $\delta(X, S)$ and $\delta(Y, S)$ are $(k-1)$ -distinguishable.
- Step 3:** If $P_k \neq P_{k-1}$, repeat Step 2, otherwise go to Step 4.
- Step 4:** Combine k^{th} equivalent sets and make them the new states of the reduced DFA.

Example

Let us consider the following DFA:



q	$\delta(q,0)$	$\delta(q,1)$
a	b	c
b	a	d
c	e	f
d	e	f
e	e	f
f	f	f

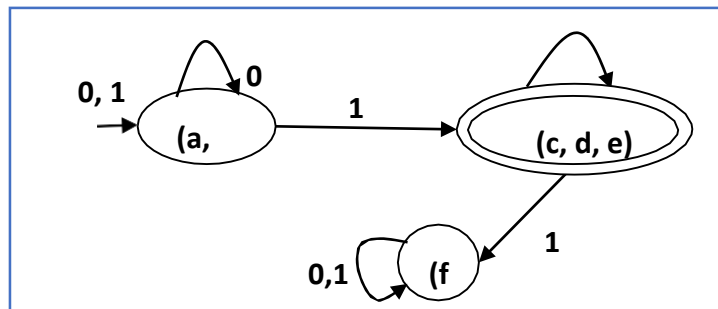
Let us apply the above algorithm to the above DFA:

- $P_0 = \{(c,d,e), (a,b,f)\}$
- $P_1 = \{(c,d,e), (a,b),(f)\}$
- $P_2 = \{(c,d,e), (a,b),(f)\}$

Hence, $P_1 = P_2$.

There are three states in the reduced DFA. The reduced DFA is as follows:

Q	$\delta(q,0)$	$\delta(q,1)$
(a, b)	(a, b)	(c,d,e)
(c,d,e)	(c,d,e)	(f)
(f)	(f)	(f)



State Table and State Diagram of Reduced DFA

Finite automata may have outputs corresponding to each transition. There are two types of finite state machines that generate output:

- Mealy Machine
- Moore machine

Moore and Mealy Machines

□

Moore and Mealy Machine

Both moore and mealy machine are special case of DFA ✓
 Both acts like o/p producers rather than language acceptors ✓
 In moore and mealy machine no need to define the final states
 No concepts of dead states and no concepts of final states
 Mealy and Moore Machines are equivalent in power.




George H Mealy
Edward F Moore

Mealy Machine

A Mealy Machine is an FSM whose output depends on the present state as well as the present input.

It can be described by a 6 tuple $(Q, \Sigma, O, \delta, X, q_0)$ where –

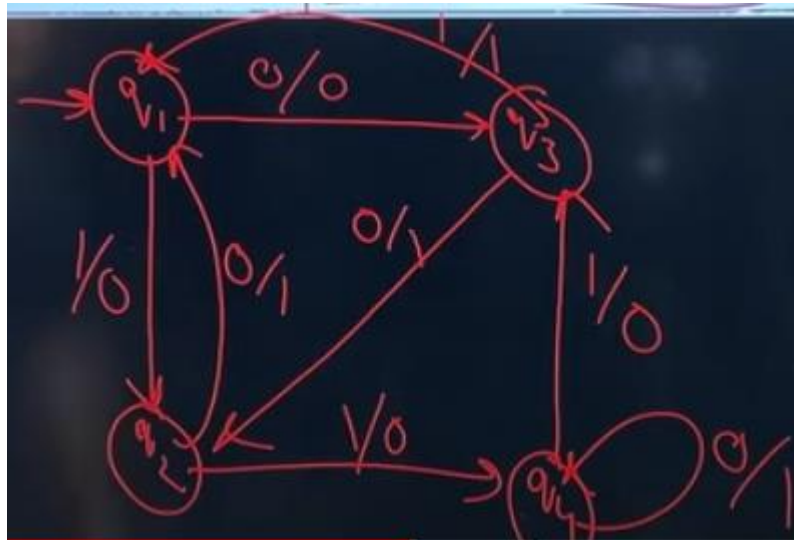
Q is a finite set of states.

- **Σ** is a finite set of symbols called the input alphabet.
- **O** is a finite set of symbols called the output alphabet.
- **δ** is the input transition function where $\delta: Q \times \Sigma \rightarrow Q$
- **X** is the output transition function where $X: Q \times \Sigma \rightarrow O$
- **q_0** is the initial state from where any input is processed ($q_0 \in Q$).

Example: The below table shows the transition table of a Mealy Machine.

Present state	Next state	
	$a = 0$	
	state	output
$\rightarrow q_1$	q_3	0
q_2	q_1	1
q_3	q_2	1
q_4	q_4	1

Next state	$a = 1$	
	state	output
	state	output
q_2	q_2	0
q_4	q_4	0
q_1	q_1	1
q_3	q_3	0



Moore Machine

Moore machine is an FSM whose outputs depend on only the present state.

A Moore machine can be described by a 6 tuple $(Q, \Sigma, O, \delta, X, q_0)$ where:

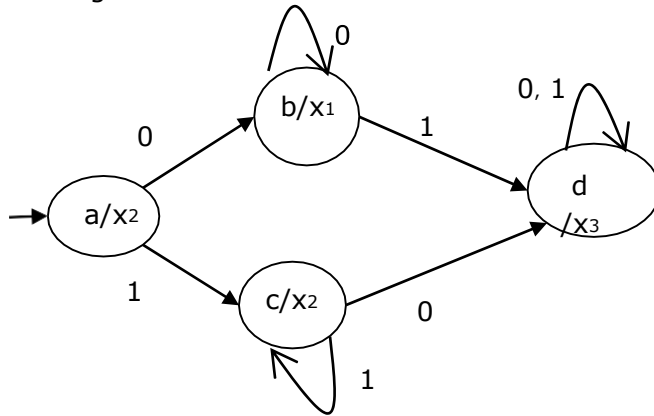
- **Q** is a finite set of states.
- **Σ** is a finite set of symbols called the input alphabet.
- **O** is a finite set of symbols called the output alphabet.
- **δ** is the input transition function where $\delta: Q \times \Sigma \rightarrow Q$
- **X** is the output transition function where $X: Q \rightarrow O$
- **q_0** is the initial state from where any input is processed ($q_0 \in Q$).

The state table of a Moore Machine is shown below –

Present State	Next State		Output
	Input = 0	Input = 1	
$\rightarrow a$	b	c	x_2
b	b	d	x_1

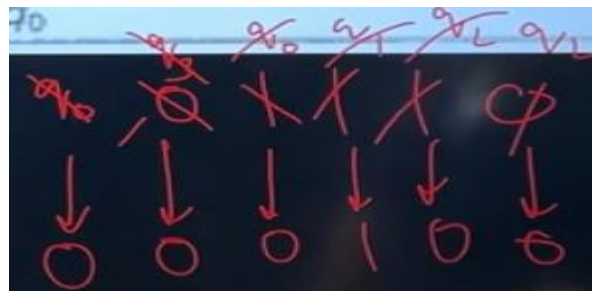
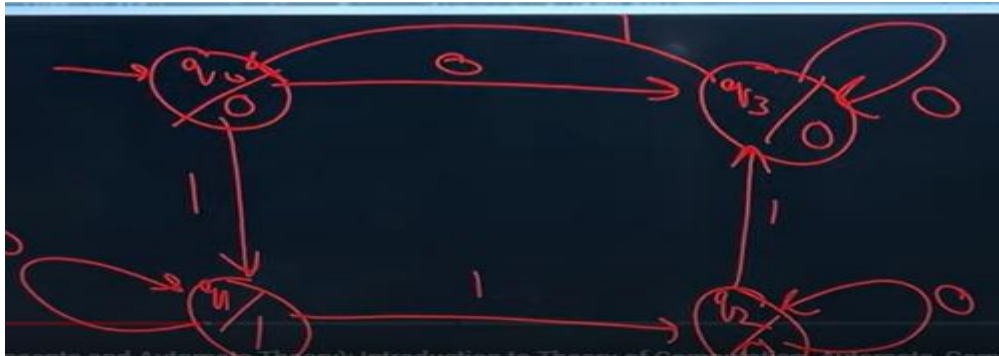
c	c	d	x_2
d	d	d	x_3

The state diagram of the above Moore Machine is:



Examples: The below table shows the transition table of a Moore Machine.

Present state	Next state δ		Output λ
	$a = 0$	$a = 1$	
$\rightarrow q_0$	q_3	q_1	0
q_1	q_1	q_2	1
q_2	q_2	q_3	0
q_3	q_3	q_0	0



Mealy Machine vs. Moore Machine

The following table highlights the points that differentiate a Mealy Machine from a Moore Machine.

Mealy Machine	Moore Machine
Output depends both upon present state and present input.	Output depends only upon the present state.
Generally, it has fewer states than Moore Machine.	Generally, it has more states than Mealy Machine.
Output changes at the clock edges.	Input change can cause change in output change as soon as logic is done.
Mealy machines react faster to inputs	In Moore machines, more logic is needed to decode the outputs since it has more circuit delays.
In mealy machine output is equal to length of string	In Moore machine output is equal to 1+length of string

Moore Machine to Mealy Machine

Algorithm 4

Input: Moore Machine

Output: Mealy Machine

Step 1 Take a blank Mealy Machine transition table format.

Step 2 Copy all the Moore Machine transition states into this table format.

Step 3 Check the present states and their corresponding outputs in the Moore Machine state table; if for a state Q_i output is m , copy it into the output columns of the Mealy Machine state table wherever Q_i appears in the next state.

Example

Let us consider the following Moore machine:

Present State	Next State		Output
	a = 0	a = 1	
⊔a	d	b	1
b	a	d	0
c	c	c	0
d	b	a	1

State table of a Moore Machine

Now we apply Algorithm 4 to convert it to Mealy Machine.

Step 1 & 2:

Present State	Next State			
	a = 0		a = 1	
	State	Output	State	Output
⊔a	d		b	
b	a		d	
c	c		c	
d	b		a	

The partial state table after steps 1 and 2

Step 3:

Present State	Next State			
	a = 0		a = 1	
	State	Output	State	Output
=> a	d	1	b	0
b	a	1	d	1
c	c	0	c	0
d	b	0	a	1

State table of an equivalent Mealy Machine

Mealy Machine to Moore Machine

Algorithm 5:

Input: Mealy Machine

Output: Moore Machine

Step 1 Calculate the number of different outputs for each state (Q_i) that are available in the state table of the Mealy machine.

Step 2 If all the outputs of Q_i are same, copy state Q_i . If it has n distinct outputs, break Q_i into n states as Q_{in} where $n = 0, 1, 2, \dots$

Step 3 If the output of the initial state is 1, insert a new initial state at the beginning which gives 0 output.

Example

Let us consider the following Mealy Machine:

Present State	Next State			
	a = 0		a = 1	
	Next State	Output	Next State	Output
a	d	0	b	1
b	a	1	d	0
c	c	1	c	0
d	b	0	a	1

State table of a Mealy Machine

Here, states 'a' and 'd' give only 1 and 0 outputs respectively, so we retain states 'a' and 'd'. But states 'b' and 'c' produce different outputs (1 and 0). So, we divide **b** into **b₀**, **b₁** and **c** into **c₀**, **c₁**.

Present State	Next State		Output
	a = 0	a = 1	
a	d	b ₁	1
b ₀	a	d	0
b ₁	a	d	1
c ₀	c ₁	c ₀	0
c ₁	c ₁	c ₀	1
d	b ₀	a	0